Citizen Assist AI — Plantão Enchente

Documentação técnica — Data Structures and Algorithms (GS 2026.2)

Integrantes: Gabriel Barbosa Furin (RM 572941) · Lucas Kiodi Moraca (RM 571004) · Renan Fracalossi Mano da Silva (RM 569610) **Linguagem:** Python 3 (somente biblioteca padrão; testado no Python 3.13).

---

1. Recorte escolhido

O sistema gerencia **serviços e orientações ao cidadão em situação de enchente**: alertas da Defesa Civil, o que fazer durante, saúde depois da enchente, FGTS e documentos. É o mesmo recorte do assistente de terminal do projeto. Os 16 registros iniciais vieram de páginas oficiais (Defesa Civil, Ministério da Saúde, FGTS e gov.br), e cada um traz a fonte e a data de verificação (04/11/2026).

2. Modelo dos registros — classe `Servico`

Campo	Tipo	Regra de validação
`id`	`int`	inteiro de 1 a 999999; **único** (duplicado é recusado)
`titulo`	`str`	obrigatório, até 100 caracteres
`categoria`	`str`	obrigatório, até 40 caracteres
`descricao`	`str`	obrigatório, até 800 caracteres
`requisitos`	`str`	obrigatório, até 640 caracteres (documentos exigidos)
`local`	`str`	obrigatório, até 160 caracteres (onde ser atendido)
`fonte`	`str`	obrigatório, até 200 caracteres

`Servico.de_dict()` valida um registro lido do arquivo e levanta `ValueError` se algo estiver errado (campo ausente, `id` que não é inteiro, texto vazio ou grande demais).

3. Estrutura de dados: lista encadeada explícita

```
        class No:
            def __init__(self, dado):
                self.dado = dado        # um objeto Servico
                self.prox = None        # referencia para o proximo No

        class ListaEncadeada:
            def __init__(self):
                self.inicio = None      # referencia para o primeiro No
                self.tamanho = 0
```

A lista **não usa** `list`, `deque` nem outra coleção pronta para guardar os registros: só nós ligados por referências (`prox`). Em Python não há `malloc`/`free`: quando um nó sai da lista e ninguém mais o referencia, o coletor de lixo libera a memória (na remoção também zeramos `atual.prox`).

Operação	Método	O que faz
Inserção	`inserir_inicio`, `inserir_fim`	recusam ID duplicado levantando `IdDuplicadoError`
Percurso	`__iter__` (e `for s in lista`)	percorre os nós com `yield`, do primeiro ao último
Busca por ID	`buscar_id`	devolve o `No` ou `None`
Pesquisa	`pesquisar`	título ou categoria que **contém** o texto, sem diferenciar maiúsculas nem acentos
Atualização	`atualizar`	troca os dados; o ID não muda; devolve `False` se não existir
Remoção	`remover`	trata primeiro, meio, último e único nó; guarda o nó anterior e religa
Ordenação	`ordenar(lista, criterio)` em `ordenacao.py`	Merge Sort que **religa os nós**

Integridade: depois de inserir, atualizar, remover ou ordenar, `tamanho` sempre bate com o número de nós. Os testes percorrem a lista nó a nó depois de cada operação e conferem a ordem dos IDs e o tamanho.

4. Organização do código

```
citizen-assist-dsa/
├── main.py            # menu (classe App)
├── servico.py         # classe Servico
├── lista.py           # classes No e ListaEncadeada
├── ordenacao.py       # Merge Sort proprio
├── analise.py         # quantidade de registros por categoria
├── armazenamento.py   # carregar e salvar em JSON
├── entrada.py         # leitura segura do teclado
├── texto.py           # normalizar (sem acentos nem maiusculas)
├── dados/servicos.json
└── testes/            # test_dsa.py, rodar_cenarios.py, entradas/ e saidas/
```

5. Menu e validação

```
 1. Cadastrar            6. Ordenar
 2. Consultar por ID     7. Listar todos
 3. Atualizar            8. Análise (quantidade por categoria)
 4. Excluir              9. Salvar em arquivo
 5. Pesquisar            0. Encerrar
```

Todas as leituras (`entrada.py`) repetem a pergunta até a entrada ser válida e nunca encerram o programa:

- letras, vazio, número fora do intervalo e número gigante → mensagem e nova pergunta;
- ID duplicado no cadastro → “Já existe um serviço com o ID …”, pede outro (0 cancela);
- campo vazio ou grande demais → mensagem e nova pergunta;
- `s/n` inválido → pergunta de novo;
- ID inexistente em consulta, atualização ou exclusão → “Nenhum serviço encontrado”;
- lista vazia → cada opção avisa que não há o que fazer;

- arquivo de dados ausente, corrompido ou com registros inválidos → o programa avisa e continua (lista vazia ou só com os registros válidos);
- fim da entrada (Ctrl+D / Ctrl+Z) ou Ctrl+C → encerra com mensagem, sem erro na tela;
- a exclusão pede confirmação; ao sair com alterações não salvas, o programa pergunta se deseja salvar.

6. Algoritmos e análise Big-O

Seja **n** o número de registros e **L** o tamanho máximo de um campo de texto (constante, no máximo 800 caracteres).

6.1 Busca

- **Por ID — busca linear** (`buscar_id`): começa em `inicio` e anda seguindo `prox` até achar o ID ou chegar ao fim (`None`). Se não achar, devolve `None` e o menu mostra "Nenhum serviço encontrado com o ID …". **Melhor caso O(1)** (primeiro nó), **pior caso e caso médio O(n)**. Numa lista encadeada não há acesso direto pela posição, por isso não dá para usar busca binária, mesmo com a lista ordenada.
- **Por título ou categoria** (`pesquisar`): percorre todos os nós e usa `contem`, que normaliza os dois textos (minúsculas, sem acentos) e procura o trecho dentro do campo. Devolve todos os resultados; se não houver nenhum, o menu avisa e sugere tentar outra palavra. Custo **O(n · L)**; como L é limitado, na prática **O(n)**.

6.2 Ordenação — Merge Sort sobre lista encadeada

1. `_dividir` acha o meio com dois ponteiros (um anda 1 nó, o outro anda 2) e corta a lista em duas.
2. `_merge_sort` ordena cada metade recursivamente.
3. `_mesclar` junta as duas metades já ordenadas, ligando sempre o menor nó primeiro.

Critérios: ID, título ou categoria (sem diferenciar maiúsculas nem acentos; desempate pelo título e depois pelo ID, então o resultado é sempre o mesmo). **Não usamos `sorted()` nem `.sort()`.**

Caso	Tempo	Espaço extra
Melhor, médio e pior	**O(n log n)**	**O(log n)** (pilha da recursão; os nós são religados, não copiados)

O tempo é O(n log n) em todos os casos porque a lista sempre é dividida ao meio (log n níveis) e cada nível percorre n nós.

6.3 Resumo das operações

Operação	Melhor	Pior	Justificativa
Inserir no início	O(n)	O(n)	ligar o nó é O(1), mas é preciso percorrer a lista para garantir que o ID não se repete
Inserir no fim	O(n)	O(n)	percorre para checar duplicado e outra vez até o último nó
Buscar por ID	O(1)	O(n)	busca linear
Pesquisar por título/categoria	O(n)	O(n)	sempre olha todos os nós
Atualizar	O(1)	O(n)	busca por ID; trocar os dados é O(1)
Remover	O(1)	O(n)	busca guardando o nó anterior; religar é O(1)
Percorrer / listar	O(n)	O(n)	visita cada nó uma vez
Ordenar (Merge Sort)	O(n log n)	O(n log n)	divisão ao meio + mesclagem
Análise por categoria	O(n)	O(n)	um passe pela lista com um dicionário (busca em dicionário é O(1) em média)
Carregar do arquivo	$O(n^2)$	$O(n^2)$	n inserções no fim, cada uma O(n)

Melhoria possível: guardar uma referência para o último nó (`fim`) deixaria o cadastro no fim sem o percurso extra, mas a checagem de ID duplicado continuaria O(n). Com n = 16 a diferença não aparece; preferimos a versão mais simples de explicar.

6.4 Análise dos dados

A opção 8 conta os registros por categoria e mostra o total, a porcentagem, uma barra de #, a categoria com mais registros e a com menos. Com os dados iniciais:

Categoria	Registros	%
Saúde depois da enchente	6	37,5
Antes e durante a enchente	4	25,0
Alertas e emergência	3	18,8
Dinheiro e documentos	3	18,8

7. Evidências de testes

7.1 Testes automáticos — `testes/test_dsa.py` (50 testes, 0 falhas)

Executar: `python -m unittest discover -s testes -v`.

Grupo	Testes	O que verifica
Lista vazia	8	buscar, remover, atualizar, percorrer, pesquisar, ordenar e analisar numa lista vazia não quebram
Inserção	4	início e fim; ordem 2,3,1; ID duplicado recusado nas duas funções; tamanho não muda
Busca	6	ID existente e inexistente; título ignorando maiúsculas e acentos; categoria; sem resultado
Atualização	3	atualiza o título; o ID não muda; inexistente devolve `False`; ordem preservada
Ordenação	7	por ID, título e categoria; sequência de ordenações; um único nó; 50 nós em ordem inversa; desempate por ID
Remoção	6	meio, primeiro, último, inexistente, esvaziar a lista e reinserir, remover duas vezes o mesmo ID
Análise	1	categorias iguais com ou sem acento contam juntas; soma das contagens = total
Arquivo	5	salvar e carregar dão os mesmos dados; arquivo inexistente; JSON corrompido; JSON que não é lista; registros ruins ignorados
Texto	2	`normalizar` e `contem`
Validação do registro	2	IDs inválidos (0, negativo, grande, texto, `None`, `True`); campo vazio ou grande demais
Leitura de entrada	6	repete até ser válido (`abc`, vazio, `1_0`, `+5`, `99`, `-1`); número gigante; texto obrigatório e limite; texto opcional; `s/n`; fim da entrada

7.2 Cenários pelo menu — `testes/rodar_cenarios.py`

Executar: `python testes/rodar_cenarios.py`. O script roda cada arquivo de `testes/entradas/` numa pasta temporária (o `dados/servicos.json` não é alterado) e grava as saídas em `testes/saidas/`.

#	Cenário	Entrada	Resultado esperado	Resultado observado
T1	Fluxo normal	listar; consultar ID 5; pesquisar título “enchente”; pesquisar categoria “saude”; pesquisar título “agua”; ordenar por título; análise	16 registros; detalhe do ID 5; resultados das buscas (a de “agua” deve achar “água”); lista em ordem alfabética; contagens por categoria	**Igual ao esperado.** Listou 16; ID 5 completo; título “enchente” → 6 resultados (IDs 5, 6, 8, 9, 12, 13); categoria “saude” → 6 (IDs 8 a 13); “agua” → ID 8 (“água segura”); ordenação começa no ID 8 e termina no ID 14; análise: 6, 4, 3 e 3 registros
T2	CRUD completo	cadastrar ID 5 (duplicado) e depois ID 100; consultar; atualizar só o título; excluir respondendo “n” e depois “s”; consultar de novo; salvar	duplicado recusado; 100 cadastrado; só o título muda; exclusão cancelada e depois feita; consulta final sem resultado; salva 16	**Igual ao esperado.** “Já existe um serviço com o ID 5”; “Serviço 100 cadastrado”; título “Título atualizado” com os outros campos mantidos; “Exclusão cancelada”; “Serviço 100 excluído”; “Nenhum serviço encontrado com o ID 100”; “16 registro(s) salvo(s)”
T3	Entradas inválidas	`abc`, `99`, vazio e `-1` no menu; `x`, `0` e um número de 20 dígitos como ID; ID 9999; escolha 3 e `xx` na pesquisa; busca `zzzz`; cadastro com ID 5, título vazio e título de 150 letras; `x` na confirmação; `z` ao sair	cada entrada recusada com mensagem, sem encerrar o programa	**Igual ao esperado.** “digite apenas números inteiros”; “fora do intervalo permitido (0 a 9)”; “Digite um número”; “fora do intervalo permitido (1 a 999999)”; “Nenhum serviço encontrado com o ID 9999”; “(nenhum resultado)”; “Campo obrigatório”; “Texto longo demais (máximo 100 caracteres)”; “Resposta inválida: digite s para sim ou n para não”
T4	Lista vazia (arquivo inexistente)	opções 7, 2, 3, 4, 5, 6, 8	cada opção avisa que a lista está vazia	**Igual ao esperado.** “Arquivo … não encontrado. Iniciando com a lista vazia”; “A lista está vazia” em todas; análise: “Não há registros para analisar”
T5	Fim da entrada no meio do cadastro	a entrada acaba depois da categoria	encerra com segurança	**Igual ao esperado.** “Entrada encerrada.” e “Até logo!”, sem erro na tela

#	Cenário	Entrada	Resultado esperado	Resultado observado
T6	Cancelar e texto enorme	cadastrar com ID 0; depois ID 400 com título de 1500 letras e um título correto em seguida	cancela; recusa o texto enorme e pede de novo	**Igual ao esperado.** “Cadastro cancelado”; “Texto longo demais”; registro 400 cadastrado e consultado
T7	Arquivo corrompido	arquivo com `{ isso nao e json`; opção 7	avisa e começa com a lista vazia	**Igual ao esperado.** “O arquivo ruim.json está corrompido. Iniciando com a lista vazia.”
T8	Registros inválidos no arquivo	6 itens: 1 válido, 1 repetido, 1 sem campos, 1 texto, 1 com ID “abc”, 1 com título vazio	carrega só o válido e avisa dos 5 ignorados	**Igual ao esperado.** “Carregados 1 registro(s) … (5 registro(s) inválido(s) ignorado(s))”

8. Limitações

- A pesquisa procura um trecho do texto; não entende sinônimos nem erros de digitação.
- Todos os dados ficam na memória; para milhares de registros seria melhor uma árvore ou uma tabela hash.
- Se o arquivo de dados estiver corrompido e a pessoa escolher “Salvar”, o arquivo ruim é substituído pela lista atual.
- Os dados são orientações gerais; valores e prazos (por exemplo, do FGTS) podem mudar. Confira sempre na fonte indicada em cada registro.